The Relay: write-up

Long version, code and logs: `README.md` (its appendix is round 2, the fixes after my first tests). Table numbers are computed from `results.csv`; figures in the text come from it or from `logs/`. "Real" = Groq's free tier; "stand-in" = my own service.

1 What I built and why

A Python/asyncio pipeline over the three papers: **classify** → **retrieve** (BM25, 1,200-character chunks with pages, plus a table's rows when they match the question) → **decide** (rewrite the query once if matches are poor; a `fetch_section` tool call with injected faults) → **draft** (must cite `[C:paper:n]` or say "The papers don't say") → **verify** (the citation exists and every number is in the cited text) → **release**. Limiter, retry loop, stand-in and load generator are mine (plain `httpx`). **The decision that shaped it:** under pressure the system does *less*, never *worse*. When a request's estimated wait (counting everyone queued) passes 40% of its remaining time, there is no model call: the person gets the best sentences quoted from the papers, with citations, or an honest "at capacity". Cost: in a burst most get quotes, not a written answer. When Groq's daily budget is gone, the same calls go to Gemini through the same check: slow (1 to 22 s to a first word) but not sloppier. "Fast" = median first token ≤ 2,500 ms, p95 ≤ 8,000 ms.

2 The real service and my stand-in

Limit	Value found	How I found it	Confidence
Requests per minute	30 (both gpt-oss models)	Only in a 429 body ("Limit 30, Used 30, Requested 1") after a 40-request burst; no header carries it (`logs/groq_probe_1794406507.jsonl`)	high
Tokens per minute	8,000	Header `x-ratelimit-limit-tokens`, confirmed by a TPM 429; refills about 133 tokens/s	high
Requests per day	1,000, refilling 1 per 86.4 s	Header `x-ratelimit-limit-requests` and its reset ("1m26.4s")	medium (never used up)
Tokens per day	200,000 on the 120B (about 85 answers); the 20B's is higher	Only in a 429 body, hit by accident ("Limit 200000, Used 199538"); no header	high (120B), low (20B)
Requests in flight	None found up to 16	4, 8, 12, 16 at once, twice each (`logs/groq_probe2_concurrency_*.jsonl`)	low
Reply when a limit is hit	HTTP 429, JSON text names the limit and numbers; `retry-after` in seconds; tokens charged as used, not `max_tokens`	The 429s above; `logs/groq_probe2_accounting_*.jsonl`	high
Time to recover	RPM 2 s; TPM seconds; TPD 648 s for a 1,962-token request (rolling refill, about 2.3 tokens/s)	429 headers and bodies; a test reproduces the 648 s	high

Gemini, the backup: 15 requests a minute (stated only in the 429 body); 3 of 30 tiny requests got a 503. **Stand-in:** same streaming API; lognormal first-token and per-token times fitted from real probes; RPM/TPM/RPD/TPD pools; 429s naming the limit only in the body; tokens charged as used. **Least like the real thing:** its 1% failure rate and its slowdown with concurrency (0.0625 per request in flight) are pessimistic assumptions: Groq returned no 5xx in 948 requests and showed no clear slowdown from 4 to 16 in flight. Groq's queue time is not modelled and the answers are canned, so it says nothing about quality.

3 Results

Real runs are tiny on purpose (about 3 answers a minute); heavy loads ran on the stand-in. Shapes: steady, burst and sustained overload at free-tier limits and 10×, plus per-minute limits **halved** and daily budget **exhausted**; users give up after 20 s. *Succeeded* = a verified written answer; *Rate-limited* = no model call because a limit was close or hit, so the person got quotes or an honest "at capacity".

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady_free_tier	stand-in	19	478.7	2.4	2233.0 / 13717.5 / 14308.9	2234.8 / 13765.5 / 14647.9
burst_free_tier	stand-in	20	4.9	250.0	162.4 / 1965.7 / 3096.2	177.3 / 1970.2 / 3139.7
overload_free_tier	stand-in	30	159.5	11.4	1270.2 / 7236.7 / 8760.0	1273.2 / 7348.3 / 8804.0
steady_per_minute_limits_halved	stand-in	14	320.1	2.7	6741.0 / 17436.4 / 17813.0	6900.6 / 17768.8 / 17979.4
burst_per_minute_limits_halved	stand-in	20	4.8	250.1	17.1 / 683.3 / 2934.3	21.7 / 687.6 / 2937.8
steady_10x_limits	stand-in	82	228.8	21.6	1293.7 / 4396.4 / 8437.6	1298.2 / 4399.9 / 8441.2
burst_10x_limits	stand-in	150	16.0	933.8	58.0 / 5373.8 / 9997.1	62.8 / 5377.8 / 9999.7
overload_10x_limits	stand-in	248	238.6	63.5	3419.0 / 7881.1 / 11088.2	3423.8 / 7884.7 / 11093.2
daily_budget_exhausted_limiter_learns	stand-in	54	138.0	23.5	37.1 / 2542.4 / 10149.2	56.2 / 2550.1 / 10169.0
daily_budget_exhausted_second_provider	stand-in	54	138.0	23.5	1190.2 / 3882.7 / 9309.5	1203.2 / 3892.0 / 9315.3
steady_real	real	3	19.4	10.7	2499.0 / 2797.1 / 2823.6	2502.2 / 2799.9 / 2826.4
burst_real	real	40	17.8	165.9	39.1 / 17092.3 / 18012.7	41.3 / 17098.5 / 18020.1
daily_budget_exhausted_real_second_provider	real	4	39.6	9.6	2307.0 / 12737.0 / 14158.8	2309.9 / 12740.1 / 14161.9

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Error %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers (USD)
steady_free_tier	89.5	10.5	0.0	0.0	0.05	3	2073.7	0.408
burst_free_tier	20.0	80.0	0.0	0.0	0.00	5	1900.2	0.377
overload_free_tier	40.0	60.0	0.0	0.0	0.03	3	1817.8	0.369
steady_per_minute_limits_halved	57.1	42.9	0.0	0.0	0.14	3	1947.1	0.394
burst_per_minute_limits_halved	10.0	90.0	0.0	0.0	0.00	3	1788.5	0.358
steady_10x_limits	100.0	0.0	0.0	0.0	0.04	4	2061.0	0.404
burst_10x_limits	28.7	71.3	0.0	0.0	0.01	62	1872.7	0.376
overload_10x_limits	81.5	18.5	0.0	0.0	0.06	10	1587.1	0.327
daily_budget_exhausted_limiter_learns	37.0	63.0	0.0	0.0	0.02	4	2084.9	0.411
daily_budget_exhausted_second_provider	74.1	25.9	0.0	0.0	0.02	4	2082.7	0.408
steady_real	100.0	0.0	0.0	0.0	0.00	2	2232.0	0.394
burst_real	12.5	87.5	0.0	0.0	0.10	5	2272.2	0.615
daily_budget_exhausted_real_second_provider	100.0	0.0	0.0	0.0	0.00	2	2161.0	0.381

Reading the tables. At 10×, steady load is fully served (p95 4,396.4 ms) and the 150-request burst now meets the target: written answers' p95 first token is 5,834.5 ms (about 13.2 s in round 1), mostly because my limiter had reserved `prompt + max_tokens`, a third more than a call cost. Overload at 10× is 81.5% written (66.5% in round 1) with p95 7,881.1 ms, but its median, 3,419.0 ms, **misses the 2,500 ms target**: most requests are written answers, not 19 ms quotes. Free-tier steady **misses the p95 target** (13,717.5 ms): one request waited 14,456.7 ms before getting quotes (169 laptop stalls over 100 ms in that run). Halved limits: steady 57.1% written (498 stalls, so read the tail as an upper bound). The real runs were tiny and slow that evening and say nothing about the policy. Cost: stand-in tokens at the 120B list price.

Quality (54 questions: 20 easy, 9 hard, 25 new in round 2, each frozen before use). Table 3 counts an answer correct only if right **and** cited on a gold page, so it is stricter than the README's rubric (G on the 20B: 56.0% here, 78.0% by rubric). Wrong answers are confident: a 120B answer read a BLEU score (38.1) as training FLOPs and passed the number check. Table 3 times exclude queueing.

4 Where the time goes

Slowest of 4 real requests (5,217.5 ms to first byte): **3,508.8 ms before my server saw it** (laptop stalls), **12.3 ms** in the limiter, **1,683.3 ms** on provider calls, 13.0 ms retrieval, **0 ms retrying**; all before the first token, since the reply is held until verified.

5 When a limit is hit

Per minute: a new request's wait is estimated counting everyone queued. Under 15% of its remaining time → normal; under 40% → a lighter written answer (3 passages, no classify call); 40% or more → no model call: quoted passages if they clearly match, else an honest "at capacity". First come, first served among what the token budget can pay for: early arrivals get written answers, late ones quotes; a request already under way is never dropped. **Per day:** the wall is known from config or learned from the first 429 body (learning it cost exactly one request); no new Groq call starts and requests go to Gemini, then to quotes or a decline.

Retrieval: quality versus cost

Setting (piece size, pieces fetched, re-rank)	Question set and drafting model	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Refused wrongly %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
A bm25 1200/200 k5 (v1)	hard9_gpt-oss-20b	9	55.6	11.1	22.2	11.1	2224	1946.8 / 5619.7	1946.8 / 5619.7
B dense 1200/200 k5 (v1)	hard9_gpt-oss-20b	9	33.3	33.3	22.2	11.1	2174	2625.6 / 67416.2	2625.6 / 67416.2
C dense+rerank 1200/200 k5 (v1)	hard9_gpt-oss-20b	9	55.6	11.1	22.2	11.1	2209	10313.9 / 79410.6	10313.9 / 79410.6
D bm25 600/100 k8 (v1)	hard9_gpt-oss-20b	9	33.3	22.2	22.2	22.2	1908	2866.6 / 4243.1	2866.6 / 4243.1
E bm25 2400/300 k3 (v1)	hard9_gpt-oss-20b	9	33.3	22.2	22.2	22.2	2416	1950.2 / 4531.8	1950.2 / 4531.8
G bm25 1200/200 k5 + gated table row (v2)	hard25_gpt-oss-20b	25	56.0	12.0	20.0	12.0	2667	2373.4 / 5177.7	2373.4 / 5177.7

Chosen: **G, BM25 with 1,200/200-character chunks, top 5, plus a table row when it shares at least 40% of the question's words.** A–E ran on 9 round-1 questions and G on 25 new ones, so the rows are not comparable; I chose G on a no-model check: it finds the answer's page for all 23 original answerable questions (v1: 22). On the v1 index, re-ranking found the gold page 100% of the time but took 8,254.6 ms (median); dense was slower and worse (91.3% vs 95.7%). B's and C's p95s are one request each (110 s and 121 s of 9; cause not established), so read their medians. **Near a limit,** top-k drops from 5 to 3 (about 40% fewer prompt tokens).

How this library will mislead a naive retriever

1. **Tables come out scrambled.** "How many parameters does the big Transformer model have?" (213M) was answered once in 6 runs; one model read the FLOPs column.
2. **Same words recur across papers.** "How many attention heads does the base Transformer use?" ranks an appendix and an ablation table first.
3. **Structure heuristics lie.** My filter treated 26 H-Net chunks as references; 25 were, and one held the start of Appendix A (fixed in v2).

6 Retries

At most 3 attempts, jittered backoff never shorter than `retry-after`, never past the deadline, never a daily-limit 429, retries capped at 20% of first attempts. **Where retrying made it worse** (stand-in, 30% failures, 110% of capacity): unbounded retries sent 277 provider calls against 228, reserved 44.4% more tokens for 4 more full answers (81 vs 77), and p95 first token more than doubled (8,530.1 vs 4,037.6 ms).

7 What the person sees

At 3 pm, with Groq's daily budget gone, the request goes to Gemini and gets a written, verified answer (40 of 54 in the stand-in run, against 20 with no second provider). If Gemini is also out, within about 20–30 ms: *"Our language-model budget for today is used up, so I can't write a summary right now. Here are the most relevant passages from the papers, quoted verbatim:"* plus citations. If nothing matched: *"Our model budget is used up for now and I couldn't find a passage that clearly answers this. Please try again in about N minutes."* N is computed by the limiter (a test reproduces Groq's real 648 s).

8 What I'm unsure about, what I left out, what I learned

Least certain: that the stand-in's heavy-load behaviour stands for real Groq: its slowdown and failure rate are assumptions, untested beyond 16 requests in flight. If wrong, the 10× tails and the retry penalty move; the policies do not. **Not built:** request hedging and a multi-process limiter. **Learned: A limit you only learn from an error body.** A provider's headers don't tell you the limits that will actually stop you. Groq's response headers gave me 8,000 tokens a minute and 1,000 requests a day. The 30-requests-a-minute limit appears in no header, only in the text of a 429 ("Limit 30, Used 30, Requested 1"). The 200,000-tokens-a-day limit on the 120B appeared the same way, after I had already used it up. That second limit is the one that matters. At about 2.3k tokens an answer it allows roughly 85 answers a day, nowhere near what 1,000 requests suggests. So I treat 429 bodies as data, and my limiter learns limits while it runs.

Feedback on Vaani

- **What stood out.** The docs say a 200 from the call API means the call was dispatched, not answered. That is an honest line between "accepted" and "done". The prompt diff in Teacher AI also keeps a person in control.
- **What confused me.** The homepage says p90 latency is about 450 ms, while the developers page gives p50 600 ms and p90 900–1,200 ms. Nothing says what is being timed or over how many calls.
- **What I'd change.** Publish one latency page with what is timed, the percentiles, the sample size and the conditions, plus what happens when a customer goes over their concurrent-call limit. I'd want that before trusting a voice agent with real callers.

How I worked with the assistant

The raw, uncleaned transcript is `docs/claude-session.jsonl` (Groq organization id replaced by `[REDACTED-GROQ-ORG-ID]`; nothing else edited), plus the planning chat, unedited, in `docs/planning-chat.md`. Full account: README section 12.

Three prompts that mattered most: (1) **read the first page and list the requirements, constraints and tradeoffs before any design**: it showed the overload policy was the central decision and led to rebuilding every table from raw logs; (2) **"What tech stack should I use for this?"**: plain asyncio and httpx, no LangChain, my own limiter, an open-loop load generator in a separate process; (3) **"Choose the groq models urself… keep in mind the rate limits so that we don't burn out our complete quota in testing itself… Gemini as secondary"**: it kept the first real runs at 4, 4 and 8 requests and made the stand-in necessary.

What the assistant got wrong (I did not catch these; it found each while checking its own results, and each is kept in `logs/`): it assumed two Llama models that returned 404; its citation parser missed 【 】 and wrongly declined 12 of 20 easy questions; its first overload rule ignored the queue, so burst users waited about 20 s; and the small "fast" model it planned for was slower (706.9 vs 420.9 ms median first token).

A decision where I overruled it: its `.gitignore` excluded the paper PDFs; I removed that rule so they are in the repository.

Distinctly mine: the instruction to protect the free quota, which shaped the test plan; delegating the model choice and naming Gemini as secondary; asking for the CI check and a README that reads like a person wrote it; reading vaani.dev and writing the "learned" and Vaani sections; reading the brief first and settling the stack in the planning chat.